1 Number Systems

1.1 Notation

When programming, we cannot use subscripts like 17_8 to denote the base.

- Binary denoted as `0b`, e.g. `0b110000011111`
- Octal (base 8) denoted as `0`, e.g. `06037`
- Decimal is without prefix, e..g `3103`
- Hex denoted as `0x`, e.g. `0xC1F`

1.2 Representing negative numbers

Two's complement: "The two's complement of an N-bit number is defined as its complement with respect to 2^N"

$$\overset{2^3}{\overline{=}}1000_2 - 0\overset{3}{\overline{=}}11_2 \;=\; 101 \quad (\text{represents } -3)$$

$$x = (x_{N-1}\, x_{N-2} \ldots x_0)_2 = \left(-x_{N-1}\, 2^{N-1} + \sum_{i=0}^{N-2} x_i\, 2^i \right)$$

Decimal	Two's comp.
-4	100
-3	101
-2	110
-1	111
0	000
1	001
2	010
3	011

Why 2's-complement? Single zero reprsentation simplifies equality checking, and arithmetic is overall easier.

1.3 Integers

- *Signed* n-bit integers can represent $-2^{n-1}, \ldots, 2^{n-1} - 1$
- *Unsigned* n-bit integer can represent $0, \ldots, 2^{n-1}$

1.4 Checking overflows in 2's complement

- (pos) + (neg) cannot overflow
- (pos) − (pos) cannot overflow
- (neg) + (neg) should be negative. Carry bit turns it positive.
- (pos) + (pos) should be positive. Carry bit turns it negative.

1.5 Extending numbers

Consider the 8-bit number 01001011_2. To extend it to 16 bits, we pad the front with 0s: 0000000001001011_2. Now the number 11001011_2: If unsigned, we pad with 0s as prior. If signed, we pad with 1s instead: 1111111111001011_2.